877. Stone Game - Explanation

Problem Link

Description

Alice and Bob are playing a game with piles of stones. There are an **even** number of piles arranged in a row, and each pile has a positive integer number of stones `piles[i]`.

The objective of the game is to end with the most stones. The total number of stones across all the piles is **odd**, so there are no ties.

Alice and Bob take turns, with **Alice starting first**. Each turn, a player takes the entire pile of stones either from the **beginning** or from the **end** of the row. This continues until there are no more piles left, at which point the person with the **most stones wins**.

Assuming Alice and Bob play optimally, return `true` if Alice wins the game, or `false` if Bob wins.

Example 1:

Input: piles = [1,2,3,1]

Output: true

Explanation: Alice takes first pile, then Bob takes the last pile, then Alice takes the pile from the end and Bob takes the last remaining pile. Alice's score is 1 + 3 = 4. Bob's score is 1 + 2 = 3.

Example 2:

Input: piles = [2,1]

Output: true

Constraints:

- `2 <= piles.length <= 500`
- `piles.length` is **even**.
- `1 <= piles[i] <= 500`
- `sum(piles[i])` is **odd**.

Topics

Company Tags

Please upgrade to NeetCode Pro to view company tags.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Dynamic Programming (Interval DP)** - Computing optimal results over ranges [l, r] with subproblems based on shrinking intervals

- **Game Theory / Two-Player Games** - Understanding how to model turn-based games where both players play optimally
- **Recursion with Memoization** - Converting recursive solutions to efficient DP by caching overlapping subproblems
- **Mathematical Reasoning** - Recognizing that structural properties (even pile count, odd total) can lead to direct solutions

---

1. Recursion

Intuition

This is a two-player game where Alice and Bob take turns picking from either end of the piles array. Both play optimally, meaning each player maximizes their own score. We can simulate this using recursion where we track the current range `[l, r]` and determine whose turn it is based on the range length. Alice wants to maximize her score, while Bob's moves affect what Alice can collect later.

Algorithm

1. Define a recursive function `dfs(l, r)` that returns Alice's maximum score for the subarray `[l, r]`.
2. If `l > r`, return `0` (no piles left).
3. Determine if it is Alice's turn: it is Alice's turn when the number of remaining piles `(r - l + 1)` is even.
4. If it is Alice's turn, she can take from either end and we add her choice to the score. Recurse and take the maximum of taking left or right.
5. If it is Bob's turn, he takes optimally but we only track Alice's score (add `0` for his pick).
6. Compare Alice's final score against Bob's (total minus Alice's score) to determine the winner.

Stone Game - Dynamic Programming with Recursion

Step 1 / 17

Initialize with piles [5, 3, 4, 5] and calculate total sum = 17

Upgrade to Pro to unlock visualizations

0.5x1x2x

```
public class Solution {
    public boolean stoneGame(int[] piles) {
        int total = 0;
        for (int pile : piles) {
            total += pile;
        }

        int aliceScore = dfs(0, piles.length - 1, piles);
        return aliceScore > total - aliceScore;
    }

    private int dfs(int l, int r, int[] piles) {
        if (l > r) {
```

```
        return 0;
    }
    boolean even = (r - l) % 2 == 0;
    int left = even ? piles[l] : 0;
    int right = even ? piles[r] : 0;
    return Math.max(dfs(l + 1, r, piles) + left, dfs(l, r - 1, piles) + right);
  }
}
```

Collapse

Time & Space Complexity

- Time complexity:
- $O(2n)$
- $O(2$
- n
- $)$
- Space complexity:
- $O(n)$
- $O(n)$

---

2. Dynamic Programming (Top-Down)

Intuition

The recursive solution has overlapping subproblems since the same `(l, r)` range can be reached through different sequences of moves. By memoizing results for each `(l, r)` pair, we avoid recomputing the same states and achieve polynomial time complexity.

Algorithm

1. Create a 2D memoization table `dp` initialized to `-1` (or use a hash map).
2. In the recursive function, first check if `dp[l][r]` has been computed. If so, return the cached value.
3. Compute the result as in the plain recursion approach.
4. Store the result in `dp[l][r]` before returning.
5. The final answer is whether `dp[0][n-1]` (Alice's score) is greater than `total - dp[0][n-1]` (Bob's score).

Stone Game - Dynamic Programming with Memoization

Step 1 / 19

Initialize with piles [5,3,4,5] and empty memoization dictionary

Upgrade to Pro to unlock visualizations

0.5x1x2x

```
public class Solution {
    private int[][] dp;
```

```
    public boolean stoneGame(int[] piles) {
        int n = piles.length;
        dp = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                dp[i][j] = -1;
            }
        }

        int total = 0;
        for (int pile : piles) {
            total += pile;
        }

        int aliceScore = dfs(0, n - 1, piles);
        return aliceScore > total - aliceScore;
    }

    private int dfs(int l, int r, int[] piles) {
        if (l > r) {
            return 0;
        }
        if (dp[l][r] != -1) {
            return dp[l][r];
        }
        boolean even = (r - l) % 2 == 0;
        int left = even ? piles[l] : 0;
        int right = even ? piles[r] : 0;
        dp[l][r] = Math.max(dfs(l + 1, r, piles) + left, dfs(l, r - 1, piles) + right);
        return dp[l][r];
    }
}
```

Expand

Time & Space Complexity

- Time complexity:
- $O(n2)$
- $O(n$
- 2

-)
- Space complexity:
- O(n2)
- $O(n$
- 2
-)

---

3. Dynamic Programming (Bottom-Up)

Intuition

Instead of recursion with memoization, we can fill the DP table iteratively. We process subproblems in order of increasing range length. For each range `[l, r]`, we compute Alice's optimal score based on already-solved smaller ranges `[l+1, r]` and `[l, r-1]`.

Algorithm

1. Create a 2D DP table of size `n x n`.
2. Iterate `l` from `n-1` down to `0` (to ensure smaller ranges are solved first).
3. For each `l`, iterate `r` from `l` to `n-1`.
4. Determine whose turn it is based on `(r - l) % 2`.
5. For base case `l == r`, Alice takes the pile if it is her turn, otherwise `0`.
6. For larger ranges, take the maximum of choosing left or right (adding the pile value only on Alice's turn).
7. Return whether `dp[0][n-1] > total - dp[0][n-1]`.

Piles

5

0

3

1

4

2

5

3

n

4

DP Table

0

1

2

3

0

0

0

0

0

1

0

0

0

0

0

2

0

0

0

0

0

```
3
0
0
0
0
```

Initialize variables: n = 4 for piles [5, 3, 4, 5]

Upgrade to Pro to unlock visualizations

0.5x1x2x

```
public class Solution {
    public boolean stoneGame(int[] piles) {
        int n = piles.length;
        int[][] dp = new int[n][n];

        for (int l = n - 1; l >= 0; l--) {
            for (int r = l; r < n; r++) {
                boolean even = (r - l) % 2 == 0;
                int left = even ? piles[l] : 0;
                int right = even ? piles[r] : 0;
                if (l == r) {
                    dp[l][r] = left;
                } else {
                    dp[l][r] = Math.max(dp[l + 1][r] + left, dp[l][r - 1] + right);
                }
            }
        }

        int total = 0;
        for (int pile : piles) {
            total += pile;
        }

        int aliceScore = dp[0][n - 1];
        return aliceScore > total - aliceScore;
    }
}
```

Expand

Time & Space Complexity

- Time complexity:
- O(n2)
- $O(n$
- 2
-)
- Space complexity:
- O(n2)
- $O(n$
- 2
-)

---

4. Dynamic Programming (Space Optimized)

Intuition

Looking at the DP transitions, `dp[l][r]` only depends on `dp[l+1][r]` and `dp[l][r-1]`. When iterating by increasing `l` from right to left and `r` from left to right, we only need values from the current row being built. This allows us to compress the 2D table into a 1D array.

Algorithm

1. Create a 1D DP array of size `n`.
2. Iterate `l` from `n-1` down to `0`.
3. For each `l`, iterate `r` from `l` to `n-1`.

4. `dp[r]` represents the previous value from `dp[l+1][r]`, and `dp[r-1]` represents `dp[l][r-1]`.
5. Update `dp[r]` with the maximum score for the current range.
6. Return whether `dp[n-1] > total - dp[n-1]`.

Stone Game - Dynamic Programming

Step 1 / 18

Initialize with piles = [5, 3, 4, 5], n = 4, and dp array of size 4 filled with zeros.

Upgrade to Pro to unlock visualizations

0.5x1x2x

```
public class Solution {
    public boolean stoneGame(int[] piles) {
        int n = piles.length;
        int[] dp = new int[n];

        for (int l = n - 1; l >= 0; l--) {
            for (int r = l; r < n; r++) {
                boolean even = (r - l) % 2 == 0;
                int left = even ? piles[l] : 0;
                int right = even ? piles[r] : 0;

                if (l == r) {
                    dp[r] = left;
                } else {
                    dp[r] = Math.max(dp[r] + left, dp[r - 1] + right);
                }
            }
        }

        int total = 0;
        for (int pile : piles) {
            total += pile;
        }

        int aliceScore = dp[n - 1];
        return aliceScore > (total - aliceScore);
    }
}
```

Expand

Time & Space Complexity

- Time complexity:
- $O(n2)$
- $O(n$
- 2
- $)$
- Space complexity:
- $O(n)$

- $O(n)$

---

5. Return TRUE

Intuition

Here is the key insight: with an even number of piles and an odd total sum, Alice can always win. She can always choose to take all even-indexed piles or all odd-indexed piles. Since the total is odd, one of these sets must have a larger sum. Alice, moving first, can force the game to give her whichever set she prefers, guaranteeing a win.

Algorithm

1. Simply return `true`.

Stone Game - Mathematical Solution

Step 1 / 4

Piles

5

0

3

1

4

2

5

3

Initialize with piles [5, 3, 4, 5]

Upgrade to Pro to unlock visualizations

0.5x1x2x

```
class Solution:
    def stoneGame(self, piles: List[int]) -> bool:
        return True
```

Time & Space Complexity

- Time complexity:
- $O(1)$
- $O(1)$
- Space complexity:
- $O(1)$
- $O(1)$

Common Pitfalls

Overcomplicating the Solution

The mathematical insight that Alice always wins (due to even pile count and odd total) means you can simply return `true`. However, many solvers implement full DP without recognizing this property. While the DP approach is valid and educational, understanding why Alice always wins demonstrates deeper problem analysis.

Confusing Whose Turn It Is

In the DP approach, determining whose turn it is based on the range `[l, r]` can be tricky. Alice moves when the number of remaining piles is even

(since the game starts with an even number and she goes first). Using `(r - l + 1) % 2 == 0` or equivalently `(r - l) % 2 == 1` indicates Alice's turn, but off-by-one errors here are common.

Wrong Comparison for Final Result

The question asks whether Alice wins (strictly greater score), not whether she ties or wins. Comparing `alice_score >= total - alice_score` instead of `alice_score > total - alice_score` gives incorrect results. Since the total is odd and all values are positive integers, a tie is impossible in this specific problem, but the comparison should still be strictly greater.